

04 — GIN Indexes (Generalized Inverted Index)

"GIN: the go-to index for multi-valued data — arrays, JSONB, full-text search."

Table of Contents

1. [Learning Objectives](#)
 2. [GIN Architecture](#)
 3. [ASCII Diagram: GIN Structure](#)
 4. [JSONB Indexing](#)
 5. [Array Indexing](#)
 6. [Full-Text Search Indexing](#)
 7. [GIN Operators Reference](#)
 8. [Pending List and Fastupdate](#)
 9. [Creating GIN Indexes](#)
 10. [EXPLAIN Output for GIN Scans](#)
 11. [GIN vs GiST for Full-Text Search](#)
 12. [pg_trgm Extension with GIN](#)
 13. [Common Mistakes](#)
 14. [Best Practices](#)
 15. [Performance Considerations](#)
 16. [Interview Questions & Answers](#)
 17. [Exercises with Solutions](#)
 18. [Production Scenarios](#)
 19. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Describe GIN's inverted index structure and how it handles multi-valued data types
 - Create GIN indexes for JSONB columns and explain the difference between `jsonb_ops` and `jsonb_path_ops`
 - Create GIN indexes for array columns and use containment operators
 - Create GIN indexes for full-text search using `tsvector` columns
 - Use `pg_trgm` with GIN for LIKE/ILIKE pattern matching
 - Explain the pending list mechanism and `fastupdate` behavior
 - Identify when GIN is preferred over GiST for full-text search
-

GIN Architecture

GIN (Generalized Inverted Index) is designed for **multi-valued data types**: a single row can contribute multiple entries to the index. Think of it like a book's index: one chapter appears on many pages, and the page index maps each word to the list of pages where it appears.

Key Concepts

- **Entry**: an atomic value extracted from a document (a word from text, a key from JSON, an element from an array)
- **Posting list**: for each entry, the list of heap ctids (rows) that contain it

- The index structure is: `entry → [ctid1, ctid2, ctid3, ...]`

Why GIN for Multi-Valued Data?

Consider indexing tags: a blog post can have 10 tags. A B-tree index could only efficiently find posts by one specific tag (with an equality condition on an array element, which standard B-tree doesn't support). GIN extracts all 10 tags as separate index entries, each pointing back to the post's ctid. A query for `tag = 'postgresql'` finds that entry and retrieves all matching ctids instantly.

ASCII Diagram: GIN Structure

GIN index on `articles(tags)` where `tags` is `TEXT[]`

Document data (heap):

Row ctid (0,1): tags = ['postgresql', 'indexing', 'performance']

Row ctid (0,2): tags = ['postgresql', 'tutorial']

Row ctid (0,3): tags = ['mysql', 'indexing']

Row ctid (0,4): tags = ['postgresql', 'indexing', 'vacuum']

GIN Index structure (inverted):

B-tree of entries (sorted)	
[indexing]————→	posting list: [(0,1),(0,3),(0,4)]
[mysql]————→	posting list: [(0,3)]
[performance]————→	posting list: [(0,1)]
[postgresql]————→	posting list: [(0,1),(0,2),(0,4)]
[tutorial]————→	posting list: [(0,2)]
[vacuum]————→	posting list: [(0,4)]

Query: `WHERE tags @> ARRAY['postgresql', 'indexing']`

1. Look up 'postgresql' → [(0,1),(0,2),(0,4)]
2. Look up 'indexing' → [(0,1),(0,3),(0,4)]
3. Intersect: [(0,1),(0,4)]
4. Fetch heap rows (0,1) and (0,4)

Result: rows with both 'postgresql' AND 'indexing' tags

Short posting list (fits in leaf page):

'postgresql' ctid=(0,1) ctid=(0,2) ctid=(0,4)

Long posting list (overflow to separate page):

'common_word' [ptr to list page]



```
| Posting tree: B-tree of ctids for 'common_word' |
| ctid=(0,1), ctid=(0,2), ... ctid=(9,999) (thousands of rows) |
```

JSONB Indexing

JSONB (binary JSON) is one of the most common GIN use cases in PostgreSQL.

Two GIN Operator Classes for JSONB

1. jsonb_ops (default)

Indexes every key, value, and key-path in the JSONB document.

```
-- Default: jsonb_ops
CREATE INDEX idx_events_data ON events USING GIN (data);
-- Equivalent to:
CREATE INDEX idx_events_data ON events USING GIN (data jsonb_ops);
```

Supported operators with jsonb_ops :

- @> containment (does document contain this sub-document?)
- ? key exists: data ? 'key'
- ?| any key exists: data ?| ARRAY['key1', 'key2']
- ?& all keys exist: data ?& ARRAY['key1', 'key2']
- @@ (jsonpath: PostgreSQL 12+)
- @? (jsonpath exists: PostgreSQL 12+)

```
-- Queries that use jsonb_ops GIN index:
SELECT * FROM events WHERE data @> '{"type": "click"}';
SELECT * FROM events WHERE data ? 'user_id';
SELECT * FROM events WHERE data ?| ARRAY['email', 'phone'];
SELECT * FROM events WHERE data ?& ARRAY['user_id', 'session_id'];
SELECT * FROM events WHERE data @? '$.user_id ? (@ > 1000)';
```

2. jsonb_path_ops

Only indexes values (not keys), specifically optimized for @> containment queries. Smaller index, faster @> queries, but fewer supported operators.

```
CREATE INDEX idx_events_data_path ON events USING GIN (data jsonb_path_ops);
```

Only supported operator: @> (containment)

```
-- Only this type of query:
SELECT * FROM events WHERE data @> '{"type": "click", "user_id": 42}';
```

JSONB Index Size Comparison

Operator Class	Supported Ops	Index Size	@> Speed
jsonb_ops	@>, ?, `?	, ?&, @@, @?`	Larger
jsonb_path_ops	@> only	~25-30% smaller	Slightly faster

Practical JSONB Examples

```
-- Setup
CREATE TABLE events (
  id BIGSERIAL PRIMARY KEY,
  created_at TIMESTAMPTZ DEFAULT now(),
  data JSONB NOT NULL
);

INSERT INTO events (data) VALUES
('{{"type": "page_view", "user_id": 1, "page": "/home", "tags": ["web", "mobile"]}}'),
('{{"type": "click", "user_id": 2, "element": "button", "tags": ["web"]}}'),
('{{"type": "purchase", "user_id": 1, "amount": 99.99, "items": ["shirt",
"pants"]}}');

-- GIN index
CREATE INDEX idx_events_gin ON events USING GIN (data);

-- Query: find all click events
SELECT * FROM events WHERE data @> '{"type": "click"}';

-- Query: find events with a specific user_id
SELECT * FROM events WHERE data @> '{"user_id": 1}';

-- Query: find events that have an 'amount' key
SELECT * FROM events WHERE data ? 'amount';

-- Query: find events with web tag in tags array
SELECT * FROM events WHERE data @> '{"tags": ["web"]}';
```

Array Indexing

GIN indexes work natively on PostgreSQL array types.

```
-- Setup
CREATE TABLE articles (
  id SERIAL PRIMARY KEY,
  title TEXT,
  tags TEXT[]
);

-- GIN index on array column
CREATE INDEX idx_articles_tags ON articles USING GIN (tags);
```

```

-- Supported array operators:
-- @> (contains): WHERE tags @> ARRAY['postgresql']
-- <@ (contained by): WHERE tags <@ ARRAY['postgresql','mysql','redis']
-- && (overlaps): WHERE tags && ARRAY['postgresql','mysql']
-- = (equals): usually B-tree is better

-- Queries:
-- Articles tagged with ALL of these tags:
SELECT * FROM articles WHERE tags @> ARRAY['postgresql', 'performance'];

-- Articles tagged with ANY of these tags:
SELECT * FROM articles WHERE tags && ARRAY['postgresql', 'mysql'];

-- Articles whose tags are a subset of these:
SELECT * FROM articles WHERE tags <@ ARRAY['postgresql', 'mysql', 'redis',
'performance'];

```

Integer Array Example

```

-- User permissions stored as integer codes
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    permissions INTEGER[]
);

CREATE INDEX idx_users_permissions ON users USING GIN (permissions);

-- Find users with admin (1) AND write (2) permissions:
SELECT * FROM users WHERE permissions @> ARRAY[1, 2];

-- Find users with any privileged permission (3,4,5):
SELECT * FROM users WHERE permissions && ARRAY[3, 4, 5];

```

Full-Text Search Indexing

GIN is the recommended index type for full-text search in PostgreSQL.

tsvector Column Approach (Recommended)

```

-- Add a pre-computed tsvector column
ALTER TABLE articles ADD COLUMN search_vector TSVECTOR;

-- Populate it
UPDATE articles SET search_vector = to_tsvector('english', title || ' ' || body);

-- Create GIN index
CREATE INDEX idx_articles_fts ON articles USING GIN (search_vector);

-- Keep it updated with a trigger

```

```

CREATE FUNCTION update_search_vector() RETURNS trigger AS $$
BEGIN
    NEW.search_vector = to_tsvector('english', NEW.title || ' ' ||
COALESCE(NEW.body, ''));
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_articles_search_vector
    BEFORE INSERT OR UPDATE ON articles
    FOR EACH ROW EXECUTE FUNCTION update_search_vector();

-- Full text search query:
SELECT title, ts_rank(search_vector, query) AS rank
FROM articles, to_tsquery('english', 'postgresql & indexing') query
WHERE search_vector @@ query
ORDER BY rank DESC
LIMIT 10;

```

Expression Index Approach

```

-- No extra column needed, but computed on-the-fly during index build
CREATE INDEX idx_articles_fts_expr ON articles
USING GIN (to_tsvector('english', title || ' ' || COALESCE(body, '')));

-- Query must use the same expression:
SELECT * FROM articles
WHERE to_tsvector('english', title || ' ' || COALESCE(body, ''))
    @@ to_tsquery('english', 'postgresql & indexing');

```

GIN Operators Reference

JSONB Operators

```

data @> '{"key": "value"}'    -- contains sub-document
data ? 'key'                  -- key exists at top level
data ?| ARRAY['k1', 'k2']      -- any key exists
data ?& ARRAY['k1', 'k2']      -- all keys exist
data @? '$.key ? (@ == "val")' -- jsonpath match exists
data @@ '$.key == "val"'       -- jsonpath match

```

Array Operators

```

arr @> ARRAY[1, 2]    -- contains all elements
arr <@ ARRAY[1, 2, 3] -- is contained by
arr && ARRAY[1, 2]     -- overlaps (any element in common)

```

Full-Text Operators

```
tsvector @@ tsquery    -- matches full text query
```

Range Operators (via GiST, but GIN also used with certain extensions)

For range types, GiST is preferred over GIN.

Pending List and Fastupdate

The Problem: GIN Write Cost

Building GIN entries is expensive — each row insertion may add many index entries (one per tag, one per JSON key, one per word). Inserting directly into the GIN tree on every write is very slow.

The Pending List Solution

GIN maintains a **pending list**: a small WAL-logged list of new entries that haven't been merged into the main GIN tree yet.

Write path:

```
INSERT row → add entries to pending list (fast, O(1))
```

Background merge (gin_pending_list_limit reached OR VACUUM):

```
pending list entries → merge into main GIN B-tree (slow, batch)
```

Read path:

```
query → search main GIN tree + search pending list → merge results
```

Fastupdate Configuration

```
-- fastupdate = on (default): use pending list for fast writes
CREATE INDEX idx_articles_tags ON articles USING GIN (tags)
WITH (fastupdate = on);

-- fastupdate = off: insert directly into GIN tree (slow writes, no pending list
search)
CREATE INDEX idx_articles_tags ON articles USING GIN (tags)
WITH (fastupdate = off);
```

When to Disable Fastupdate

- When query latency consistency is critical (pending list search adds variable cost)
- When the table is mostly read (few writes, so pending list rarely fills)
- When vacuum is infrequent and pending list grows large

```
-- Check pending list size
SELECT pg_size_pretty(pg_relation_size('idx_articles_tags'));
```

```
-- Force cleanup of pending list
SELECT gin_clean_pending_list('idx_articles_tags'::regclass);
```

Creating GIN Indexes

```
-- JSONB (default jsonb_ops)
CREATE INDEX CONCURRENTLY idx_products_attrs ON products USING GIN (attributes);

-- JSONB with jsonb_path_ops (smaller, @> only)
CREATE INDEX CONCURRENTLY idx_products_attrs_path ON products
USING GIN (attributes jsonb_path_ops);

-- Array
CREATE INDEX CONCURRENTLY idx_posts_tags ON posts USING GIN (tags);

-- tsvector column
CREATE INDEX CONCURRENTLY idx_articles_search ON articles USING GIN (search_vector);

-- Expression (compute tsvector on-the-fly)
CREATE INDEX CONCURRENTLY idx_articles_search_expr ON articles
USING GIN (to_tsvector('english', title));

-- With fastupdate disabled
CREATE INDEX CONCURRENTLY idx_articles_tags_nfu ON articles
USING GIN (tags) WITH (fastupdate = off);

-- Partial GIN index
CREATE INDEX CONCURRENTLY idx_active_events ON events
USING GIN (data) WHERE status = 'active';
```

EXPLAIN Output for GIN Scans

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM articles WHERE tags @> ARRAY['postgresql'];
```

```
Bitmap Heap Scan on articles  (cost=12.00..234.56 rows=45 width=128)
    (actual time=0.234..1.234 rows=42 loops=1)
    Recheck Cond: (tags @> '{postgresql}'::text[])
    Heap Blocks: exact=38
    Buffers: shared hit=54
    -> Bitmap Index Scan on idx_articles_tags
        (cost=0.00..11.99 rows=45 width=0)
        (actual time=0.189..0.189 rows=42 loops=1)
        Index Cond: (tags @> '{postgresql}'::text[])
Planning Time: 0.234 ms
Execution Time: 1.456 ms
```


GIN searches always produce a **Bitmap Index Scan** (not a direct Index Scan), which is then fed into a Bitmap Heap Scan. This is because GIN collects all matching ctids first, then fetches heap pages in order.

GIN vs GiST for Full-Text Search

Feature	GIN	GiST
Build time	Slower	Faster
Index size	Larger	Smaller
Query speed	Faster (for exact lookup)	Slower
Update speed	Faster (pending list)	Slower
Best for	Mostly-static, query-heavy data	Frequently-updated data
Supports tsquery ranking?	No (use tsvector column + rank function)	No

Rule of thumb: Use GIN for full-text search when data is mostly static or when query speed is more important than index build/update speed.

pg_trgm Extension with GIN

Trigram indexing enables efficient `LIKE '%middle%'` and `ILIKE` queries that B-tree cannot support.

```
-- Install extension
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- GIN index on text column for LIKE/ILIKE
CREATE INDEX idx_products_name_trgm ON products
USING GIN (name gin_trgm_ops);

-- Queries that now use the index:
SELECT * FROM products WHERE name LIKE '%laptop%';
SELECT * FROM products WHERE name ILIKE '%DELL%';
SELECT * FROM products WHERE name ~ 'dell|lenovo'; -- regex!

-- Similarity search
SELECT name, similarity(name, 'laptop') AS sim
FROM products
WHERE name % 'laptop' -- similarity operator (pg_trgm)
ORDER BY sim DESC;
```

How Trigrams Work

A trigram is a 3-character sequence: 'hello' → 'he', 'hel', 'ell', 'llo', 'lo '

For `LIKE '%hello%'`, PostgreSQL extracts trigrams from the search pattern and uses the GIN index to find rows that contain all those trigrams, then rechecks the full LIKE.

Common Mistakes

1. Using jsonb_ops when only @> queries are needed

```
-- Wastes space and build time
CREATE INDEX USING GIN (data);
-- Better for @>-only workload:
CREATE INDEX USING GIN (data jsonb_path_ops);
```

2. Using GIN on columns with very high cardinality and no repeated values

GIN is only efficient when entries appear in multiple rows (shared posting lists). On a unique text column, GIN gives no benefit — every entry points to exactly one row.

3. Forgetting to use the right operators

```
-- Does NOT use GIN index (wrong operator)
SELECT * FROM articles WHERE 'postgresql' = ANY(tags);
-- Use GIN-supported operator instead:
SELECT * FROM articles WHERE tags @> ARRAY['postgresql'];
```

4. Large pending list causing slow queries

If autovacuum is not running frequently enough, the pending list grows large, making every GIN query slower. Monitor and tune autovacuum for GIN-indexed tables.

5. Not using tsvector column for repeated full-text searches

Computing `to_tsvector()` on every query is expensive. Pre-compute and store it.

Best Practices

- JSONB:** Use `jsonb_path_ops` if only `@>` queries; use `jsonb_ops` if you also need `?`, `?|`, `?&` key existence checks.
 - Full-text search:** Pre-compute and store `tsvector` in a separate column, index that column with GIN, and update it via trigger.
 - pg_trgm:** Create both a GIN and a B-tree index on text columns that are searched both by equality and by LIKE patterns. Use GIN for LIKE, B-tree for equality.
 - Fastupdate:** Leave `fastupdate = on` (default) for write-heavy tables. Disable only if query latency consistency is critical.
 - Monitor pending list:** If you notice GIN queries gradually slowing over time, check the pending list and tune autovacuum's `autovacuum_vacuum_scale_factor` for the table.
 - Partial GIN indexes:** For JSONB or array columns where only some rows are queried, use partial indexes to reduce index size.
-

Performance Considerations

GIN Index Build Time

GIN indexes take significantly longer to build than B-tree because each row contributes multiple entries. Use `maintenance_work_mem` to speed up builds:

```
SET maintenance_work_mem = '2GB';
CREATE INDEX CONCURRENTLY idx_articles_search ON articles USING GIN (search_vector);
```

GIN Index Size

GIN indexes can be as large as or larger than the table itself for full-text search or dense JSONB documents. Monitor with:

```
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS idx_size,
    pg_size_pretty(pg_relation_size(indrelid)) AS tbl_size
FROM pg_stat_user_indexes
WHERE indexname LIKE '%gin%';
```

Partial GIN for JSONB (Huge Size Savings)

```
-- Only index active events (80% reduction in index size for tables with old data)
CREATE INDEX idx_active_events_data ON events
USING GIN (data) WHERE status != 'archived';
```

Interview Questions & Answers

Q1. What is an inverted index and why is GIN called "Generalized Inverted Index"?

A: An inverted index maps from data content to the documents that contain it — the reverse of a regular index which maps from document to content. "Generalized" means it works with any data type that can be decomposed into multiple elements, not just text. A book's index is a classic inverted index: each word maps to the page numbers where it appears. GIN applies this concept to arrays (each element maps to containing rows), JSONB (each key/value maps to containing rows), and text (each word maps to containing rows).

Q2. What is the difference between `jsonb_ops` and `jsonb_path_ops` for GIN?

A: `jsonb_ops` (default) indexes every key, value, and key-value pair in the JSONB document, supporting `@>`, `? , ?| , ?& , @@ ,` and `@?` operators. `jsonb_path_ops` indexes only values (not keys separately) and only supports the `@>` containment operator. The result: `jsonb_path_ops` produces a 20-30% smaller index and slightly faster `@>` queries, but cannot answer key-existence queries like `data ? 'field_name'`. Choose `jsonb_path_ops` when you only need containment queries; use `jsonb_ops` when you also need to check key existence.

Q3. What is the GIN pending list and why does it exist?

A: The pending list is a WAL-logged buffer of recently-added GIN entries that have not yet been merged into the main GIN B-tree structure. It exists because inserting into the main GIN tree for every write is expensive (each row can add many entries). Instead, new entries are appended to the pending list (fast $O(1)$ operation), and periodically (by vacuum or when the list exceeds `gin_pending_list_limit`) they are bulk-merged into the main tree. Query plans check both the main tree and the pending list, merging results. This `fastupdate` optimization dramatically improves write throughput at the cost of slightly slower queries when the pending list is large.

Q4. Why do GIN queries always produce Bitmap Index Scans rather than regular Index Scans?

A: GIN collects posting lists (sets of ctids) for all matching entries, then must intersect or union them to find rows matching the full query (e.g., rows containing ALL of the specified tags). This naturally produces a bitmap of matching ctids. A regular Index Scan would need to merge results from multiple postings lists on the fly in heap page order, which is less efficient. The Bitmap Index Scan collects all ctids, sorts them by heap page, and then the Bitmap Heap Scan reads heap pages in order (maximizing sequential I/O).

Q5. How would you implement case-insensitive full-text search in PostgreSQL?

A: Use `to_tsvector()` which automatically lowercases and stems words:

```
-- Store tsvector (inherently case-insensitive)
ALTER TABLE articles ADD COLUMN search_vector TSVECTOR
    GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body)) STORED;
CREATE INDEX ON articles USING GIN (search_vector);

-- Query is also case-insensitive:
SELECT * FROM articles
WHERE search_vector @@ plainto_tsquery('english', 'PostgreSQL Performance');
-- Finds 'postgresql', 'PostgreSQL', 'POSTGRESQL' equally
```

Q6. When would you use pg_trgm with GIN instead of a regular B-tree or full-text search?

A: Use `pg_trgm` GIN when you need to search for arbitrary substrings (`LIKE '%middle%'` or `ILIKE '%pattern%'`), which neither B-tree (can only do prefix matching) nor `tsvector` full-text search (works on word tokens, not arbitrary substrings) can handle. `pg_trgm` decomposes strings into 3-character sequences (trigrams) and stores them in a GIN index. Typical use cases: product name search with substring matching, autocomplete with partial matching from the middle of words, fuzzy matching with the `%` similarity operator.

Q7. What is the `&&` operator for arrays and how does it differ from `@>` ?

A: `&&` is the "overlaps" operator — it returns true if the two arrays share at least one element. `@>` is "contains" — it returns true only if the left array contains all elements of the right array. Example:

`ARRAY[1,2,3] @> ARRAY[1,2]` is true (contains both 1 and 2), while `ARRAY[1,2,3] && ARRAY[2,4]` is true (shares element 2) but `ARRAY[1,2,3] @> ARRAY[2,4]` is false (does not contain 4). For "find rows tagged with ANY of these tags" use `&&` ; for "find rows tagged with ALL of these tags" use `@>` .

Exercises with Solutions

Exercise 1

Design a GIN index strategy for a `products` table with a `metadata JSONB` column. Queries include both: `WHERE metadata @> '{"brand": "Nike"}'` and `WHERE metadata ? 'discount'`.

Solution:

```
-- Need jsonb_ops because both @> and ? operator are used
CREATE INDEX CONCURRENTLY idx_products_metadata
ON products USING GIN (metadata);
-- (jsonb_ops is the default, explicitly: USING GIN (metadata jsonb_ops))

-- Both queries will use this index:
EXPLAIN SELECT * FROM products WHERE metadata @> '{"brand": "Nike"}';
EXPLAIN SELECT * FROM products WHERE metadata ? 'discount';
```

Exercise 2

A `documents` table has a `content TEXT` column. Enable efficient substring search (e.g., `WHERE content ILIKE '%contract%'`) and word-based full-text search.

Solution:

```
-- Install extensions
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Trigram GIN for substring search
CREATE INDEX CONCURRENTLY idx_docs_content_trgm
ON documents USING GIN (content gin_trgm_ops);

-- Full-text search tsvector column + GIN
ALTER TABLE documents ADD COLUMN search_vec TSVECTOR
    GENERATED ALWAYS AS (to_tsvector('english', content)) STORED;
CREATE INDEX CONCURRENTLY idx_docs_fts ON documents USING GIN (search_vec);

-- Substring search (uses trgm index):
SELECT * FROM documents WHERE content ILIKE '%contract law%';

-- Word/phrase search (uses tsvector index):
SELECT * FROM documents
WHERE search_vec @@ plainto_tsquery('english', 'contract law');
```

Production Scenarios

Scenario 1: E-Commerce Product Attribute Search

A marketplace with 2M products, each with a `JSONB attributes` column storing brand, color, size, material, etc. Users filter by multiple attributes simultaneously.

```
-- Use jsonb_path_ops (all queries are @> containment)
CREATE INDEX CONCURRENTLY idx_products_attrs ON products
USING GIN (attributes jsonb_path_ops);
```

```
-- Multi-attribute filter:
SELECT id, name, price
FROM products
WHERE attributes @> '{"brand": "Nike", "color": "black", "size": "L"}'
ORDER BY price;
-- Uses GIN index: finds intersection of all three attribute posting lists
```

Scenario 2: Multi-Tenant Event Analytics

An event tracking system stores events as JSONB. Each tenant queries their own events filtered by event type.

```
-- Partial GIN index per tenant status
CREATE INDEX CONCURRENTLY idx_events_recent ON events
USING GIN (payload)
WHERE created_at > NOW() - INTERVAL '90 days';

-- Query recent events with specific type:
SELECT count(*), payload->>'event_type'
FROM events
WHERE created_at > NOW() - INTERVAL '30 days'
      AND payload @> '{"source": "mobile"}'
GROUP BY payload->>'event_type';
```

Cross-References

- [01_index_fundamentals.md](#) — Index types overview
- [05_gist_indexes.md](#) — GiST for comparison (geometric, range types)
- [../08_Query_Optimization/03_scan_types.md](#) — Bitmap Index Scan
- [../08_Query_Optimization/05_query_planner.md](#) — Planner statistics for complex types